Looped Threat Context Reasoning for Scalable Multiagent Target Defense

First Author, Second Author, and Third Author

***Abstract*—Multiagent target defense requires defenders to protect several assets from attackers. Because team and asset counts are rarely known in advance, a policy should train once on small engagements and transfer to larger ones. Transfer is hard because an attacker's threat depends on the target it approaches and on the teammates already covering it. We call this relation the threat context. We propose Looped Entity Attention with Fused reading (LEAF), which builds this context by looping one weight-shared entity-attention operator. A query formed from the defender's current state reads every round, the reads are fused, and recurrent memory enters only after reading. Training samples the number of rounds, so the policy can act after any round. In the SkyHAD aerial defense environment, policies trained with at most 10 defenders and 3 targets transfer without retraining to 50 defenders or 12 targets. LEAF has the lowest target damage in every test configuration, and in every test group even its weakest training run beats the best run of each baseline. Ablations confirm the roles of context construction, multi-round refinement, and memory placement. With a single round of reasoning, LEAF keeps nearly all of its protection, whereas a fixed-depth model collapses.**

***Note to Practitioners*—Teams of autonomous aircraft that guard airports, ports, or industrial sites must handle attacks whose size is unknown in advance. This work provides one control policy that is trained in small simulated engagements and then runs unchanged with several times more defenders, attackers, and protected sites. Every defender runs a copy of the same network on the positions, velocities, and status of the aircraft and sites it observes, and the network returns its next maneuver. Adding aircraft or sites needs no retraining, because all entities share the same parameters. The network first relates each attacker to the protected sites and nearby defenders, then interprets this picture from the defender's own situation, and only afterward uses its memory of earlier steps. Operators can also set how many reasoning rounds the network runs. A single round already keeps nearly all of the protection, so the controller can be matched to the onboard computer and the required decision rate without training a new model. The policy has so far been tested in simulation, where the defenders see every aircraft and site and the attackers follow a fixed rule. Field use calls for further tests with onboard sensors, communication delays, and attackers that adapt their tactics.**

***Index Terms*—Multiagent reinforcement learning, multi-target defense, scale generalization, entity attention, iterative reasoning, unmanned aerial vehicles.**

I. Introduction

DEFENDING stationary assets with a team of autonomous vehicles is a central problem for multirobot systems and aerial swarms [1]–[4]. The numbers of defenders, attackers, and protected targets are rarely known before a mission, so training a policy for every combination is impractical. A deployable controller should be trained once on small engagements and then run without retraining on much larger ones. We study this zero-shot scale generalization [5], [6].

Manuscript received Month DD, 2026. This work was supported by ...
The authors are with ... (e-mail: ...).

Placeholder for Fig. 1, single column.
The figure plan is given in the source comments.

Fig. 1. Threat context and the design of LEAF. (a) The same attacker approaching the same target is far more urgent when no teammate guards the target, so its own features cannot reveal its threat. (b) Where relations, memory, and depth enter. A single-read policy queries the entities once without relating them. An entity Transformer relates them in a fixed stack that also carries the recurrent memory. LEAF relates them with a weight-shared loop whose depth is set at deployment, reads every round with a current-state query, and lets memory enter only after the read.

Scale generalization is hard in target defense because threat is relational. Air-defense practice assesses a threat by its capability against the defended assets [7], and defense games show that the outcome depends on how defenders are arranged around the targets [2], [8]. The urgency of an attacker therefore depends on its motion, on the target it approaches, and on the teammates already covering that target. We call this joint relation the threat context. In Fig. 1(a), the same attacker approaches the same target in two scenes, and teammates guard the target only in the first. The features of the attacker are identical in both scenes, so no function of them can separate the two. A defender has to relate attackers to targets and targets to teammates before it decides [9], [10]. Larger engagements produce these relations in new combinations, so the computation that builds them must be shared across entities and remain valid for sets of any size [11], [12].

The context must also be read in the right way. The same context calls for different responses from a free defender and from one already engaged elsewhere, so each defender should

read it from its own state, as agent-specific attention does [6], [13]. Memory is needed as well, because attackers choose their targets privately and the scene is only partially observable [14], [15]. A recurrent state, however, moves into unseen regions when its inputs differ from those of training [16], [17], and a larger engagement supplies such inputs. If memory takes part in building or reading the threat context, this drift distorts the reading of the current scene.

A last question is how much relational reasoning a decision needs. One round of attention relates entities in pairs, while a guarded target is a relation among an attacker, its target, and nearby teammates. Further rounds compose such relations from earlier ones, as multiple hops do in memory networks [9], [18]. The required depth varies across inputs [19], [20], and a network trained at one depth degrades when it runs with fewer layers [20], [21]. A defense policy with a fixed depth therefore cannot shorten its reasoning to meet a tighter time budget.

Existing methods meet only part of these requirements. Assignment-based defenders, from differential games to learned allocation, compress the threat context into discrete defender-attacker pairs [4], [8], [22], [23]. Value factorization methods such as QMIX and DCG improve how utilities combine [24], [25], while each utility still comes from one read of the scene. Entity-based methods such as REFIL and SPECTra accept any team size [6], [26], yet they read the entities once without relating them first, as the top row of Fig. 1(b) shows. Entity Transformers such as TransfQMix relate entities over several layers [27], [28], but the same stack carries the recurrent memory and has a depth fixed in training.

Our key insight is that a defender's decision separates into three steps with different inputs. Building the threat context needs only the current entities. Reading it needs only the defender's current state. Inferring the hidden intent of attackers is the only step that needs history. A policy that generalizes across scales should give each step its own module, let history enter last, and train the context builder to work at any depth.

We propose LEAF (Looped Entity Attention with Fused reading). At each decision, LEAF refines the current entities with one weight-shared entity-attention operator applied in a loop, following the shared-depth computation of Universal and looped Transformers [29], [30]. A query formed only from the defender's current state reads the entities after every round, and learned weights fuse these reads. A recurrent network then combines the fused feature with the defender's memory and outputs action values. Training samples the number of rounds, so that every round count yields a usable policy [21]. Parameters are shared across entities, defenders, and rounds, and LEAF keeps the value factorization of REFIL for centralized training [6]. Unlike REFIL and SPECTra, LEAF relates entities before any defender reads them. Unlike TransfQMix, its scene processing never sees the recurrent state, and its depth can change after training, as the bottom row of Fig. 1(b) shows.

We evaluate LEAF in the SkyHAD aerial defense environment [31]. Policies trained with at most 10 defenders and 3 targets are deployed without retraining to up to 50 defenders or 12 targets. As teams grow, every learned baseline falls behind a rule-based assignment controller. LEAF has the lowest damage in every test configuration and outperforms all baselines by a wide margin. In every test group, even its weakest training run beats the best run of each baseline, and a baseline enlarged to the size of LEAF remains far behind. Single-factor ablations confirm that context construction, multi-round refinement, and memory placement each reduce damage, most clearly under extrapolation. A probe of the recurrent state confirms that memory drifts with scale while the current self state does not. Sampled-depth training keeps every execution depth usable. A fixed-depth model collapses when it runs with fewer rounds, whereas LEAF keeps nearly all of its protection with a single round.

The main contributions of this paper are as follows.

1) *Diagnosis.* We identify the threat context as the information on which zero-shot scale generalization in multi-target defense depends. We trace the scale failure of learned defenders to two design choices: reading entities before relating them, and letting a memory that drifts with scale take part in reading the scene.
2) *Method.* We propose LEAF, which separates context construction, current-state reading, and memory, and trains one weight-shared loop across sampled depths. A single policy then serves any team size and any execution depth.
3) *Evidence.* Trained on small engagements, LEAF transfers to teams and target sets several times larger with the lowest damage in every test configuration, and it keeps nearly all of its protection with a single round of reasoning.

Section II reviews related work, Section III formulates the problem, Section IV presents LEAF, Section V reports the experiments, and Section VI concludes the paper.

II. Related Work

We group prior work by the mechanism it uses to coordinate a changing number of defenders. Each family is represented among the baselines of Section V.

A. Assignment-Based Target Defense

Differential games derive defense strategies with guarantees, often by solving pairwise subgames and combining them through assignment [1], [8], [22], [32]. Task allocation makes the division of effort explicit [33]–[35]. Learning methods keep an allocation layer and learn when and how to assign, as in ALMA and hierarchical swarm defense [3], [4], [23], [36]. Our rule-based baseline re-solves an integer assignment of defenders to predicted attacker positions at every step. An assignment compresses the threat context into discrete pairs, which must be revised whenever attackers switch targets. LEAF keeps the context as entity representations and outputs maneuvers without an assignment.

B. Value Factorization and Explicit Coordination

Value decomposition connects individual utilities to a team objective through additive, monotonic, or pairwise structure [24], [25], [37]. These methods decide how utilities

combine, while each utility still comes from one read of the scene by its own agent. LEAF keeps a monotonic mixer for training and changes how the agent network forms its utilities.

C. Entity-Based Generalization Across Team Sizes

Set encoders and permutation-invariant networks process each entity with shared parameters [11], [12], [38], attention lets each agent weigh others by relevance [13], and population curricula reach large teams by training on them [39], [40]. REFIL factorizes values over random entity groups so that learned interactions recombine in new teams [6], and SPECTra pairs self-query attention with a mixer that scales with the team [26]. Both read the entity set once with the agent's own query, so relations among the other entities are not formed before the read. LEAF inserts a looped construction of these relations before the read.

D. Entity Transformers, Iterative Reasoning, and Memory

Entity Transformers apply several attention layers over entity tokens [27], [28]. UPDeT and TransfQMix carry history as a recurrent token in every layer, and their depth is fixed in training. Weight-shared iteration turns depth into repeated computation with constant parameters [19], [29], [30]. Stochastic depth, layer dropout, and early exits train networks that remain useful at several depths [20], [21], [41], [42], and jumping knowledge combines intermediate representations [43]. Recurrent memory is standard under partial observability [15], [44]. These ideas come from settings where test inputs resemble training inputs. LEAF combines a weight-shared loop, sampled depths, and fused reads inside one decision of a multiagent policy, and it keeps memory out of the loop.

III. Problem Formulation

Placeholder. Drafted in sections/sec_3.

IV. Method

Placeholder. Drafted in sections/sec_4.

V. Experiments

Placeholder. Drafted in sections/sec_5_6.

VI. Conclusion

Placeholder. Drafted in sections/sec_5_6.

References

[1] E. Garcia, D. W. Casbeer, and M. Pachter, "Design and analysis of state-feedback optimal strategies for the differential game of active defense," *IEEE Transactions on Automatic Control*, vol. 64, no. 2, pp. 553–568, 2019.

[2] D. Shishika, J. Paulos, and V. Kumar, "Cooperative team strategies for multi-player perimeter-defense games," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 2738–2745, 2020.

[3] Z. Zhao, J. Chen, B. Xin, L. Li, K. Jiao, and Y. Zheng, "Learning scalable task assignment with imperative-priori conflict resolution in multi-UAV adversarial swarm defense problem," *Journal of Systems Science and Complexity*, vol. 37, no. 1, pp. 369–388, 2024.

[4] Q. Wu, K. Liu, L. Chen, and J. Lü, "Hierarchical reinforcement learning for swarm confrontation with high uncertainty," *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 8630–8644, 2025.

[5] R. Kirk, A. Zhang, E. Grefenstette, and T. Rocktäschel, "A survey of zero-shot generalisation in deep reinforcement learning," *Journal of Artificial Intelligence Research*, vol. 76, pp. 201–264, 2023.

[6] S. Iqbal, C. A. S. de Witt, B. Peng, W. Boehmer, S. Whiteson, and F. Sha, "Randomized entity-wise factorization for multi-agent reinforcement learning," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 139, 2021, pp. 4596–4606.

[7] J. N. Roux and J. H. van Vuuren, "Threat evaluation and weapon assignment decision support: A review of the state of the art," *ORiON*, vol. 23, no. 2, pp. 151–187, 2007.

[8] E. Garcia, D. W. Casbeer, A. Von Moll, and M. Pachter, "Multiple pursuer multiple evader differential games," *IEEE Transactions on Automatic Control*, vol. 66, no. 5, pp. 2345–2350, 2021.

[9] P. W. Battaglia, J. B. Hamrick, V. Bapst *et al.*, "Relational inductive biases, deep learning, and graph networks," arXiv:1806.01261, 2018.

[10] V. Zambaldi, D. Raposo, A. Santoro *et al.*, "Deep reinforcement learning with relational inductive biases," in *International Conference on Learning Representations*, 2019.

[11] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Poczos, R. Salakhutdinov, and A. J. Smola, "Deep sets," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 3391–3401.

[12] J. Lee, Y. Lee, J. Kim, A. Kosiorek, S. Choi, and Y. W. Teh, "Set transformer: A framework for attention-based permutation-invariant neural networks," in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 97, 2019, pp. 3744–3753.

[13] S. Iqbal and F. Sha, "Actor-attention-critic for multi-agent reinforcement learning," in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 97, 2019, pp. 2961–2970.

[14] F. A. Oliehoek and C. Amato, *A Concise Introduction to Decentralized POMDPs*, ser. SpringerBriefs in Intelligent Systems. Cham: Springer, 2016.

[15] M. Hausknecht and P. Stone, "Deep recurrent Q-learning for partially observable MDPs," in *AAAI Fall Symposium on Sequential Decision Making for Intelligent Agents*, 2015.

[16] A. Lamb, A. Goyal, Y. Zhang, S. Zhang, A. Courville, and Y. Bengio, "Professor forcing: A new algorithm for training recurrent networks," in *Advances in Neural Information Processing Systems*, vol. 29, 2016, pp. 4601–4609.

[17] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, "Scheduled sampling for sequence prediction with recurrent neural networks," in *Advances in Neural Information Processing Systems*, vol. 28, 2015, pp. 1171–1179.

[18] S. Sukhbaatar, A. Szlam, J. Weston, and R. Fergus, "End-to-end memory networks," in *Advances in Neural Information Processing Systems*, vol. 28, 2015, pp. 2440–2448.

[19] A. Graves, "Adaptive computation time for recurrent neural networks," arXiv:1603.08983, 2016.

[20] Y. Kaya, S. Hong, and T. Dumitras, "Shallow-deep networks: Understanding and mitigating network overthinking," in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 97, 2019, pp. 3301–3310.

[21] A. Fan, E. Grave, and A. Joulin, "Reducing transformer depth on demand with structured dropout," in *International Conference on Learning Representations*, 2020.

[22] M. Chen, Z. Zhou, and C. J. Tomlin, "Multiplayer reach-avoid games via pairwise outcomes," *IEEE Transactions on Automatic Control*, vol. 62, no. 3, pp. 1451–1457, 2017.

[23] S. Iqbal, R. Costales, and F. Sha, "ALMA: Hierarchical learning for composite multi-agent tasks," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 7155–7166.

[24] T. Rashid, M. Samvelyan, C. Schroeder, G. Farquhar, J. Foerster, and S. Whiteson, "QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 4295–4304.

[25] W. Boehmer, V. Kurin, and S. Whiteson, "Deep coordination graphs," in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 119, 2020, pp. 980–991.

[26] H. Park, B. Seong, and S.-K. Ko, "Scalable multi-agent reinforcement learning with permutation-free networks," *IEEE Access*, vol. 14, pp. 95 650–95 664, 2026.

[27] S. Hu, F. Zhu, X. Chang, and X. Liang, "UPDeT: Universal multi-agent reinforcement learning via policy decoupling with transformers," in *International Conference on Learning Representations*, 2021.
[28] M. Gallici, M. Martin, and I. Masmitja, "TransfQMix: Transformers for leveraging the graph structure of multi-agent reinforcement learning problems," in *Proceedings of the 22nd International Conference on Autonomous Agents and Multiagent Systems*, 2023, pp. 1679–1687.
[29] M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, and L. Kaiser, "Universal transformers," in *International Conference on Learning Representations*, 2019.
[30] A. Giannou, S. Rajput, J. yong Sohn, K. Lee, J. D. Lee, and D. Papailiopoulos, "Looped transformers as programmable computers," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 11 398–11 442.
[31] "SkyHAD," Software, version 3.0.0, 2026, [Online]. Available: https://github.com/Sailero/SkyHAD.
[32] R. Yan, X. Duan, Z. Shi, Y. Zhong, and F. Bullo, "Matching-based capture strategies for 3D heterogeneous multiplayer reach-avoid differential games," *Automatica*, vol. 140, p. 110207, 2022.
[33] B. P. Gerkey and M. J. Matarić, "A formal analysis and taxonomy of task allocation in multi-robot systems," *The International Journal of Robotics Research*, vol. 23, no. 9, pp. 939–954, 2004.
[34] H.-L. Choi, L. Brunet, and J. P. How, "Consensus-based decentralized auctions for robust task allocation," *IEEE Transactions on Robotics*, vol. 25, no. 4, pp. 912–926, 2009.
[35] A. Miele, M. Lippi, and A. Gasparri, "A distributed framework for integrated task allocation and safe coordination in networked multi-robot systems," *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 11 219–11 238, 2025.
[36] X. Li, Z. Li, X. Yang, X. Yu, and J. Qiu, "Multi-agent target defense differential game: A hierarchical recognition-allocation-execution learning approach," *International Journal of Robust and Nonlinear Control*, vol. 35, no. 16, pp. 6868–6879, 2025.
[37] P. Sunehag, G. Lever, A. Gruslys *et al.*, "Value-decomposition networks for cooperative multi-agent learning based on team reward," in *Proceedings of the 17th International Conference on Autonomous Agents and Multiagent Systems*, 2018, pp. 2085–2087.
[38] J. Hao, X. Hao, H. Mao *et al.*, "Boosting multiagent reinforcement learning via permutation invariant and permutation equivariant networks," in *International Conference on Learning Representations*, 2023.
[39] Q. Long, Z. Zhou, A. Gupta, F. Fang, Y. Wu, and X. Wang, "Evolutionary population curriculum for scaling multi-agent reinforcement learning," in *International Conference on Learning Representations*, 2020.
[40] W. Wang, T. Yang, Y. Liu *et al.*, "From few to more: Large-scale dynamic multiagent curriculum learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 5, 2020, pp. 7293–7300.
[41] G. Huang, Y. Sun, Z. Liu, D. Sedra, and K. Q. Weinberger, "Deep networks with stochastic depth," in *Computer Vision – ECCV 2016*, ser. Lecture Notes in Computer Science, vol. 9908. Cham: Springer, 2016, pp. 646–661.
[42] S. Teerapittayanon, B. McDanel, and H. T. Kung, "BranchyNet: Fast inference via early exiting from deep neural networks," in *Proceedings of the 23rd International Conference on Pattern Recognition*, 2016, pp. 2464–2469.
[43] K. Xu, C. Li, Y. Tian, T. Sonobe, K. Kawarabayashi, and S. Jegelka, "Representation learning on graphs with jumping knowledge networks," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 5453–5462.
[44] T. Phan, F. Ritz, P. Altmann *et al.*, "Attention-based recurrence for multi-agent reinforcement learning under stochastic partial observability," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 27 840–27 853.